Embedded Data Structure
(Lecture Note 18)

Searching Algorithms (1)

2025. 5. 15

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class (1/3)

- **We learned about advanced sorting algorithms**
  - **Key points : divide and conquer + tree structure**

Quick sort (complexity - O(NlogN)) :

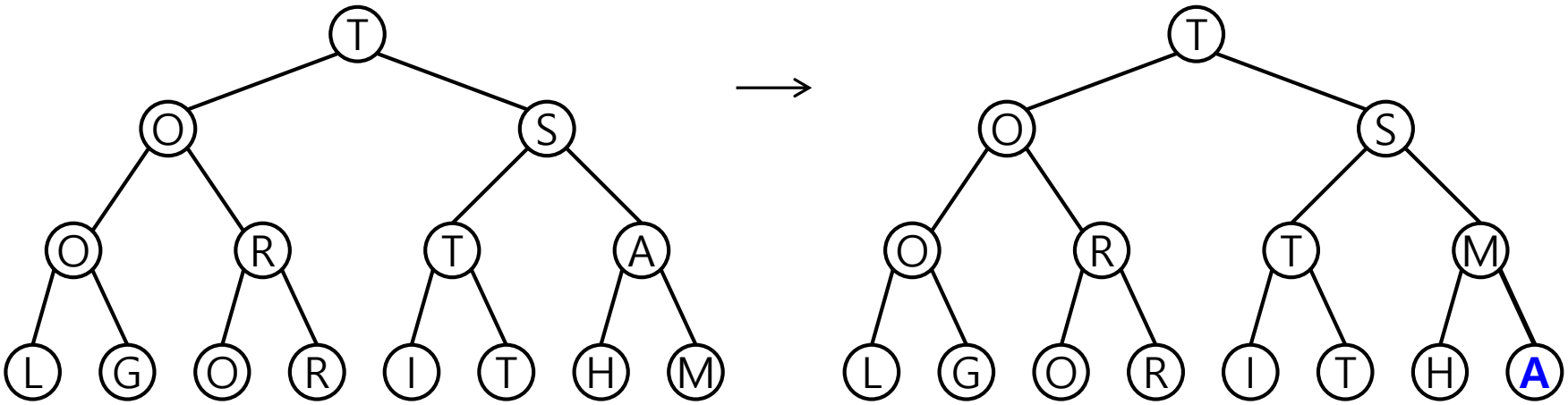
- Recursive process (dependent on the position of the pivot)
- Improved versions using random pivot and insertion sort
- Remember the generalized formulation

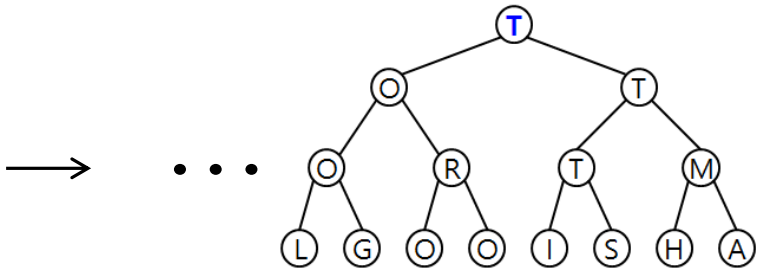
Heap sort (complexity – O(NlogN)) :

- Tree structure with the array (balanced tree)
- Two main functions : upheap and downheap
- Difference btw. top-down and bottom-up approach

Review of the Last Class (2/3)

- **Bottom-up heap construction**
  - **Make the binary tree using input data in a sequential way**
  - **Start with the end of internal nodes and conduct downheap**





You just conduct downheap twice !

Review of the Last Class (3/3)

- **Implementation of heap sorting (bottom-up)**
  - **Function for the bottom-up heap sorting**

```
void heap_sort(int *a, int N)
{
    int k, t;

    for( [          ] )
        downheap(a, N, k);

    while(N>1){
        [          ]
        [          ]
        a[N] = t;
        [          ]
    }
}
```

Loop for downheap

Root ← max value

Assign the value of end to the root

N-- and then conduct downheap

Overview of Today's Class (1/2)

- **Basic procedure of searching algorithms**
  - **Find a record using a given key value in a specific data structure**

1. My funny valentine
2. Autumn leaves
3. All the things you are
4. Cheek to cheek
5. Be witched
…

==========
Search : Autumn leaves

Title : Autumn leaves
Artist : Bill evans
…

[My music list]

Data management system includes :

1. Initialize
2. Search (using a specific key (e.g., name, …))
3. Insert
4. Delete
5. Join (optional)
6. Sort (optional)

➡ **Think about functions of linked list**

e.g., delete_node ← find and delete

Overview of Today's Class (2/2)

- **Components of searching algorithms**
  - **Searching = initialize + insert + delete + search**

    Why are they required for searching ?

  - **Two basic searching algorithms : sequential and binary search**
    - Sequential search : using unsorted data

```
for(j = 0; j<10; j++){
    if(a[j] == 3){
        find_index = j;
        break;
    }
}
```

➡ **We already conduct this searching !**

    - Binary search : using sorted data

      → It similarly performs with the divide and conquer

Binary Search (1/8)

- **Simple and fast search algorithm**
  - **Search with sorted data ($\log_2 N$ search length)**

    : Check first the median value and conduct divide and conquer

    ※ When you conduct insert or delete, keep the sorted data structure !

Pseudo code :

1. While (size of interval > 0)
    1. Find the median value on this interval
    2. If(key == median) return that index;
    3. If(median < key) select the right interval and go to step 1.
    4. Else select the left interval and go to step 1.
2. If the size of interval is 0, then key is not here

Binary Search (2/8)

- **Simple example**

  - **Procedure to find "M" in a sorted data as follows :**

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
S1	A	A	A	C	E	E	G	H	H	I	L	L	M	N	O	O	R	R	R	S	T	T
S2	**A**	A	A	C	E	E	G	H	H	I	L	L	M	N	O	O	R	R	R	S	T	**T**
S3	A	A	A	C	E	E	G	H	H	I	L	**L**	M	N	O	O	R	R	R	S	T	**T**
S4	A	A	A	C	E	E	G	H	H	I	L	**L**	M	N	O	**O**	R	R	R	S	T	T
S5	A	A	A	C	E	E	G	H	H	I	L	**L**	**M**	N	O	O	R	R	R	S	T	T
S6	A	A	A	C	E	E	G	H	H	I	L	L	**M**	N	O	O	R	R	R	S	T	T

1. Median index = (0+21)/2 = 10 → Since M > L, select the right interval

2. Median index = (11+21)/2 = 16 → Since M < R, select the left interval

⋮

➔ It repeats such operations until the size of interval is 1

※ Worst case ~ $\log_2 N$

Binary Search (3/8)

▪ C implementation

- **Think about the previous pseudo code**

```
int bi_search(int key, int *a, int N)
{
    int mid;
    int left = 0;
    int right = N-1;
    while(right >= left){          → Check the size of interval
        mid = (right + left) / 2;
        if(key == a[mid]) return mid;
        if(k < a[mid])             → Select the left interval
            right = mid – 1;
        else
            left = mid + 1;
    }
    return -1;
}
```

Binary Search (4/8)

- **C implementation – cont'd**
  - **Insert and delete (with keeping the structure of the given data)**

```
int bi_insert(int key, int *a, int *N)
{
    int p = 0;
    int i;
    while(key > a[p] && p < *N)
        p++;
    for(i = *N; i>p; i--)
        a[i] = a[i-1];
    a[p] = key;
    (*N)++;

    return p;
}
```

```
int bi_delete(int key, int *a, int *N)
{
    int p;
    int i;
    if(*N > 0){
        if((p = bi_search(key, a, [ ])<0)
            return -1;
        for(i = p+1; i<*N; i++)
            a[i-1] = a[i];
        (*N)--;
        return p;
    }
    return -1;
}
```

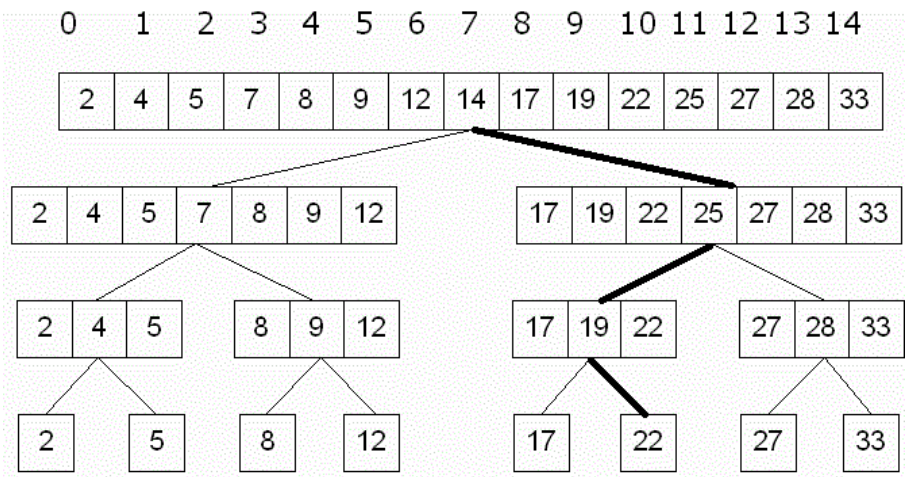
Binary Search (5/8)

- **C implementation – cont'd**
  - **Check the result of binary search**

```
void main(){
    char str[20] = "TOSORTALGORITHM";
    int size = 15;

    // Conduct sorting a str array (e.g., quick / heap sort)
    heap_sort(str, size);

    // Find a specific character using binary search
    int p = bi_search('S', str, size);
    printf("Find your key at %d (value %c)₩n", p, str[p]);

    // Insert and delete a specific character
    p = bi_insert('Z', str, &size); // or p = bi_delete('A', str, &size);
    printf("%s₩n", str);
}
```

Binary Search (6/8)

- **Characteristics of binary search**
  - **Average search length : $\log_2 N$ (cf. N ← sequential search)**
    - Divide and conquer strategy → very fast
    - There is no memory waste (even using the array structure)
    - It is very useful for the sorted(fixed) data
    - However, it requires the sorted data (and keep the given data structure)



Binary search can be interpreted by using the concept of binary tree

Binary Search (7/8)

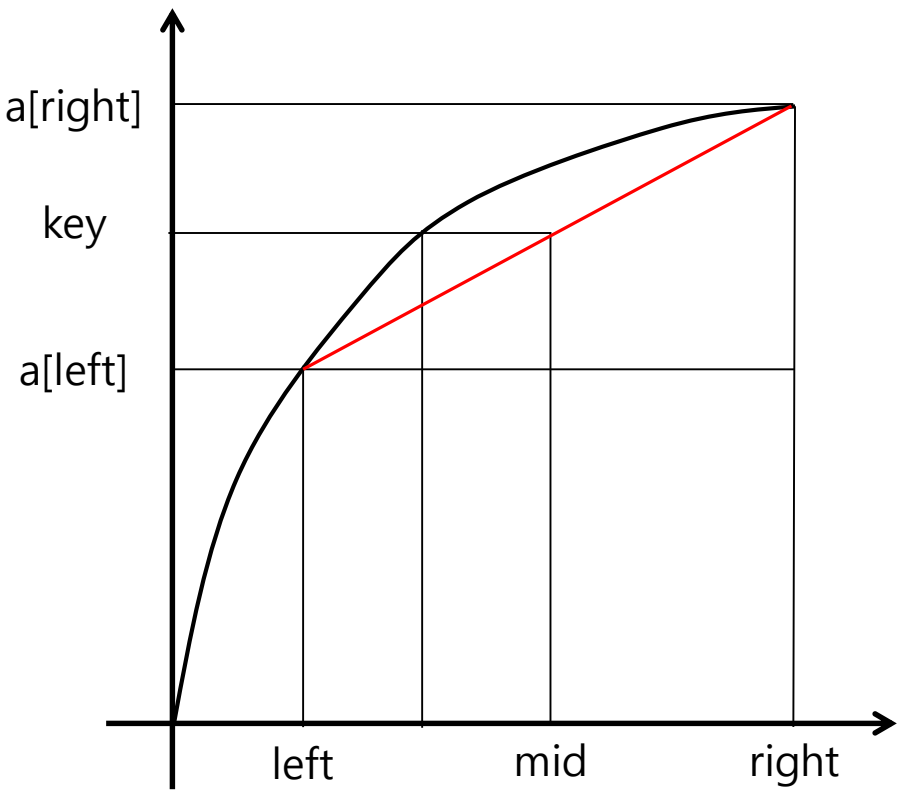
- **Improved version : interpolation**
  - **Estimate the mid. point using the interpolation**

2 100 105 459 478 701

(uneven distribution)

1 2 3 4 5 6 7 8 9 10

(even distribution)



r-l : mid-l = a[r] – a[l] : key – a[l]

mid = l + (r-l)*(key-a[l])/(a[r]-a[l])

It is effective for even distribution with numbering data

Binary Search (8/8)

- **C implementation of interpolation**
  - **Using the type casting for divisor**

```
int bii_search(int key, int *a, int N)
{
    int mid;
    int left = 0;
    int right = N-1;
    while(right >= left){
        mid = left + (float)(key-a[left])*(right-left) / (a[right] – a[left]);
        if(key == a[mid]) return mid;
        if(k < a[mid])
            right = mid – 1;
        else
            left = mid + 1;
    }
    return -1;
}
```

Summary

- **Search a specific key in your data structure**
  - **Main elements : init. / insert / delete / search**
  - **Binary search (simple scheme)**

    : using sorted data (keep the order of the data structure)